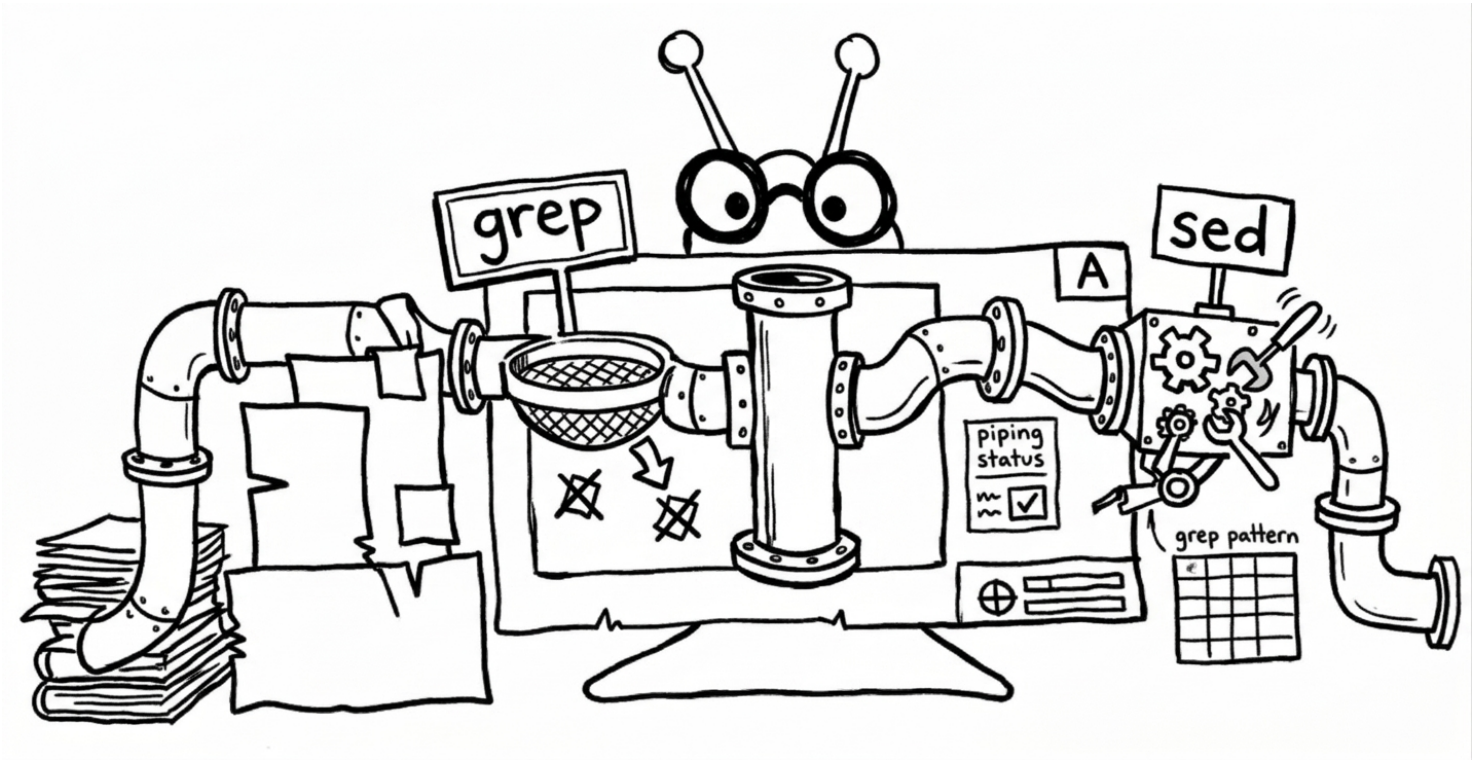
Chapitre 5: Outils de Traitement de Texte



Objectifs du chapitre

1. Construire des expressions régulières avec métacaractères, classes de caractères, quantificateurs et ancres
2. Distinguer les trois dialectes **BRE**, **ERE** et **PCRE** et savoir lequel choisir
3. Rechercher avec grep et ses options de récursivité, de contexte, de comptage et d'inversion
4. Appliquer sed pour l'édition de flux : substitution, adressage, suppression, édition en place
5. Concevoir un pipeline complet de traitement de texte, du filtrage à l'agrégation

Section 1 : Expressions régulières — fondements

définition → métacaractères → classes → quantificateurs → regroupement → avidité → exemples (IPv4, e-mail, téléphone) → BRE/ERE/PCRE → batterie de motifs

Définition : l'expression régulière

- **Expression régulière (regex)** : un motif composé de caractères littéraux et de métacaractères qui définit un ensemble de chaînes à reconnaître
- **Caractère littéral** : une lettre ou un chiffre qui ne correspond qu'à lui-même (par exemple `HWaddr` ne correspond qu'à « `HWaddr` »)
- **Métacaractère** : un symbole spécial qui active une règle de correspondance avancée
- Une regex correspond à toute chaîne qui contient le motif — `HWaddr` correspond aussi à « The `HWaddr` is unknown »
- À ne pas confondre avec le globbing du shell (*.txt), qui est un autre langage
- **Les regex décrivent du texte ; elles ne se limitent pas aux noms de fichiers**

Métacaractères : ancres et joker (^ $.)

- **.** **(point)** : un caractère quelconque, sauf le saut de ligne — **a.z** reconnaît **a2z**, **abz**, **aQz**
- **^** **(accent circonflexe)** : ancre le motif au début de la ligne
- **$** **(dollar)** : ancre le motif à la fin de la ligne
- **^$** : une ligne vide, rien entre le début et la fin

Les ancres ne correspondent à aucun caractère, elles indiquent une position

- **^[^#]** correspond à toute ligne qui ne commence pas par **#**

Classes de caractères ([ ], [^], [:digit:])

- **[abc]** (expression entre crochets) : un caractère parmi ceux listés — **b[aeiou]g** reconnaît **bag**, **beg**, **big**, **bog**, **bug**
- **[a-z]** (plage) : un caractère dans l'intervalle — **a[2-4]z** reconnaît **a2z**, **a3z**, **a4z**
- **[^abc]** (négation) : un caractère qui n'est pas listé
- Classes **POSIX** correspond qu'à un seul caractère à la fois:
  - **[:digit:]** (chiffres 0-9), **[:alpha:]** (lettres),
  - **[:alnum:]** (lettres et chiffres), **[:space:]** (espaces et tabulations)
- À l'intérieur des crochets, la plupart des métacaractères perdent leur sens spécial

Quantificateurs (* + ? {n,m})

- `*` **(astérisque)** : zéro ou plus d'occurrences de l'élément précédent
- `+` **(plus)** : une ou plus d'occurrences — l'élément doit apparaître au moins une fois
- `?` **(point d'interrogation)** : zéro ou une occurrence — élément optionnel
- `{n}` : exactement n occurrences ; `{n,}` : n ou plus ; `{n,m}` : entre n et m
- `.*` : n'importe quelle sous-chaîne, y compris vide — le motif fourre-tout
- `A.*` **Lincoln** reconnaît « **Abe Lincoln** » comme « **Abraham Lincoln** »

Regroupement, alternance et échappement (( ), |,)

- **|** **(barre verticale)** : alternance — **car|truck** reconnaît **car ou truck**
- **( )** **(parenthèses)** : regroupent une sous-expression pour y appliquer un opérateur
- (twain\.example\.com|bronto\.pangaea\.edu) : l'un ou l'autre nom d'hôte complet
- \ (antislash) : échappe un métacaractère pour le traiter littéralement
- \. reconnaît un point réel, pas n'importe quel caractère
- Le shell interprète aussi | et * : protégez toujours la regex entre guillemets

Avidité : gourmand vs paresseux

- **Correspondance gourmande (greedy)** : le quantificateur absorbe le plus de caractères possible
- **Correspondance paresseuse (lazy)** : le quantificateur absorbe le moins de caractères possible
- `*?` et `+?` : rendent un quantificateur paresseux
- Avec `<b>gras</b>` et `<b>autre</b>`, le motif `<.*>` glouton reconnaît toute la ligne
- Le motif `<.*?>` paresseux reconnaît chaque balise individuellement
- Comportement par défaut : gourmand

Exemple : adresse IPv4

- Une adresse IPv4 est composée de quatre nombres de 1 à 3 chiffres, séparés par des points
  - Exemple: 142.251.98.94
- **[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}** : quatre groupes de 1 à 3 chiffres
- Le point est échappé (**\.**) pour ne reconnaître que le séparateur littéral
- Les accolades **{1,3}** limitent chaque octet à **1 à 3 chiffres**
- **Piège** : ce motif accepte aussi des valeurs invalides comme 999.999.999.999

Exemple : adresse e-mail

- Une adresse e-mail est composée d'une partie locale, d'un @, d'un domaine et d'un **TLD** (.com, .org, .ma)
    1. **Partie locale** : [A-Za-z0-9._%+-]+ —> **lettres**, **chiffres** et **quelques symboles**
        - **+** « Le caractère ou le groupe précédent doit apparaître **1 fois ou plus**. »
    2. **@ littéral** : @
    3. **Domaine** : [A-Za-z0-9.-]+ —> **lettres**, **chiffres**, **points et tirets**
    4. **Extension** : \.[A-Za-z]{2,} —> **un point** puis **au moins deux lettres**
- **Motif complet** : [A-Za-z0-9._%+-]+@ [A-Za-z0-9.-]+\.[A-Za-z]{2,}

Exemple : numéro de téléphone

- **Regex d'un numéro de téléphone**: **\+?[0-9]{1,3}[-]?[0-9]{3}[-]?[0-9]{6}**
- **Décomposition pas à pas (ex: +33 612-345639) :**
  1. **\+?** Correspond au **+** (il est présent ici, mais grâce au **?**, il aurait pu être absent). (**+33** ou **33**)
  2. **[0-9]{1,3}** Correspond au **33** (2 chiffres, ce qui respecte la règle d'avoir entre 1 et 3 chiffres).
  3. **[-]?** Correspond à l'**espace** entre le **33** et le **612** (séparateur optionnel).
  4. **[0-9]{3}** Correspond au **612** (exactement 3 chiffres exigés par **{3}**).
  5. **[-]?** Correspond au **tiret** entre le **612** et le **345639** (séparateur optionnel).
  6. **[0-9]{6}** Correspond au **345639** (exactement **6** chiffres exigés par **{6}**).

BRE, ERE, PCRE : trois dialectes

- **BRE** (**Basic Regular Expression**) : dialecte historique où `+`, `?`, `{ }`, `( )`, `|` sont littéraux, sauf échappés avec: `\+`, `\?`, `\{\}`, `\(\)`, `\|`.
- **ERE** (**Extended Regular Expression**) : `+`, `?`, `{ }`, `( )`, `|` sont des métacaractères sans antislash
- **PCRE** (**Perl-Compatible**) : ajoute **lookahead** `(?=)`, **lookbehind** `(?<=)`
- **grep** utilise **BRE** par défaut, **ERE** avec l'option `-E`, **PCRE** avec `-P`
  - ex: `grep 'ERROR\|FATAL'` (**BRE**) équivaut à `grep -E 'ERROR|FATAL'` (**ERE**)
- **sed** utilise **BRE** par défaut

Le choix du dialecte dépend de l'outil et de ses options

Exercice : batterie de motifs à reconnaître

Pour chaque motif, dites ce qu'il reconnaît dans du texte :

- `^[0-9]{5}$`
- `https?://`
- `(cat|dog)s?`
- `colou?r`
- `\b[A-Z][a-z]+\b`
- `[A-Za-z]+[0-9]*`

Corrigé : batterie de motifs

- `^[0-9]{5}$` : une ligne composée exactement de cinq chiffres (ex: un code postal, **40140**)
- `https?://` : « http:// » ou « https:// » — le **s** est optionnel
- `(cat|dog)s?` : cat, cats, dog, dogs — alternance plus pluriel optionnel
- `colou?r` : « color » ou « colour » — le u est optionnel
- `\b[A-Z][a-z]+\b` : un mot isolé commençant par une majuscule, comme « **Bonjour** »
- `[A-Za-z]+[0-9]*` : un mot suivi d'éventuels chiffres, comme « **login42** »

Section 2 : grep (Global Regular Expression Print)

définition → options essentielles → options avancées →
trois variantes → exemples (404, IP, config, code, /etc) → grep en pipeline

Définition : grep, la recherche de motifs

- **grep** : outil qui recherche dans un flux ou des fichiers les lignes correspondant à une regex et imprime les lignes trouvées
- **Syntaxe** : grep [options] regexp [fichiers…]
- Sans fichier, grep lit l'entrée standard — parfait en pipeline (ex: ps aux | grep forkdemo)
- Par défaut, la recherche est **sensible à la casse** et utilise le dialecte **BRE**
- Le nom vient de « **g**/**re**/**p** » = **chercher globalement**, **imprimer les lignes**
- Pour les fichiers binaires, grep signale seulement que le fichier correspond

Options essentielles (-i -n -v -c -l)

- `-i` (**ignore case**) : recherche insensible à la casse — `grep -i error log.txt`
- `-n` (**line number**) : préfixe chaque ligne par son numéro
- `-v` (**invert**) : affiche les lignes qui ne correspondent pas au motif
- `-c` (**count**) : affiche le nombre de lignes correspondantes, pas les lignes
- `-l` (**files-with-matches**) : n'affiche que les noms de fichiers contenant le motif
- Combinaisons : `grep -in error *.log` pour chercher « `error` » partout, sans casse, numéroté

Options avancées (-r -E -o -w -f -A -B -C)

- -r (**recursive**) : descend récursivement dans les répertoires — grep -r eth0 /etc
- -E (**extended**) : active les expressions régulières étendues (ERE)
- -o (**only-matching**) : n'affiche que la partie correspondante, pas toute la ligne
- -w (**word**) : ne correspond qu'à des mots entiers — évite « **error** » dans « **terror** »
- -f fichier : lit les motifs depuis un fichier, un par ligne
- -A, -B, -C **n** : affichent **n** lignes de contexte après, avant, ou autour

grep, egrep, fgrep : trois variantes

- `grep` : la commande de base, en **BRE** par défaut
- `egrep` : équivaut à **grep -E**, expressions étendues par défaut
- `fgrep` : équivaut à **grep -F**, motif littéral sans aucune regex
- `fgrep` est rapide et sûr pour chercher des chaînes contenant des métacaractères
- `grep -F '$PATH'` cherche la chaîne littérale `$PATH`, sans interpréter le `$`
- Les trois partagent les mêmes options de recherche

Guillemets Simples (' ') vs Doubles (" ")

Guillemets Simples ' ':

- Protection totale / Chaîne littérale.
- Le Shell bloque le remplacement de la variable.
- **Exemple:**

  fgrep '$PATH' fichier.txt

- **Ce qui est recherché :**

  Le mot texte **exact** $PATH

Guillemets Doubles " ":

- Remplacement dynamique.
- Le Shell remplace la variable par sa valeur système.
- **Exemple:**

  fgrep "$PATH" fichier.txt

- **Ce qui est recherché :**

Le **contenu** de la variable (ex: /usr/bin:/bin).

Exemple : compter les erreurs 404

- Un journal d'accès web (Apache) contient une ligne par requête, avec le code de statut
  - **Compter les erreurs 404** : `grep -c " 404 " access.log`
  - **Chercher sans casse** : `grep -i "not found" access.log`
  - **Trouver une date précise** : `grep "12/Oct/2026" access.log`
  - **Afficher avec contexte** : `grep -A2 " 404 " access.log`
  - Le motif `" 404 "` avec espaces évite de confondre 404 avec 4040

Exemple : extraire les IP avec grep -oE

- **-o** n'imprime que la partie qui correspond, **-E** active les regex étendues
- **Extraire toutes les IP** : `grep -oE '[0-9]{1,3}(\.[0-9]{1,3}){3}' access.log`
- **Compter les IP uniques** : `grep -oE '...' access.log | sort | uniq | sort -r`
- **Top 10** : `... | head -10`
- Sans **-o**, on obtiendrait des lignes entières, pas les seules adresses

Exemple : filtrer un fichier de config

Les fichiers de configuration mélangent commentaires, lignes vides et paramètres actifs

- **Écarter les commentaires** : `grep -v '^#' /etc/ssh/sshd_config`
- **Écarter les lignes vides** : `grep -v '^$'`
- **Les deux à la fois** : `grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'`
- **Ne garder qu'un paramètre** : `grep '^Port' /etc/ssh/sshd_config`
- **Insensible à la casse** : `grep -i '^hostname' /etc/hostname`

Exemple : chercher dans du code (grep -rn)

- **Chercher récursivement** : `grep -rn "TODO" src/` — tous les fichiers, avec numéros de ligne
- **Chercher une fonction** : `grep -rn "int main" src/`
- **Ignorer la casse** : `grep -rni "error" src/`
- **Chercher un mot entier** : `grep -rnw "val" src/` — évite « val » dans « value »
- **N'afficher que les fichiers** : `grep -rl "licence" .`

Exemple : grep sur /etc

/etc regroupe les fichiers de configuration système, terrain classique de grep

- **Chercher partout un mot** : `grep -r eth0 /etc` — le périphérique réseau
- **Lister les comptes root** : `grep '^root' /etc/passwd`
- **Trouver les shells** : `grep '/bin/bash' /etc/passwd`
- **Voir les services actifs** : `grep -v '^#' /etc/services | grep -v '^$'`
- Le **LPIC-1** illustre `grep -r eth0 /etc` comme recherche type en administration

Exemple : grep dans les pipelines (ps ax | grep)

- **Chercher un processus** : `ps aux | grep bash`
- **Écarter les processus de l'utilisateur** : `ps aux | grep -v "$USER"`
- **Compter les processus d'un utilisateur** : `ps aux | grep -c "^ilias"`
- **Filtrer une configuration** : `fdisk -l | grep 'Disk /dev/sd'`
- Le motif peut aussi s'ancrer avec `^` et `$` comme vu à la section 1

Section 3 : sed (Stream Editor)

définition → substitution → adressage → suppression/impression → insertion → édition en place → exemples (CSV, config, CRLF, shebang)

Définition : sed, l'éditeur de flux

- **sed** : éditeur non interactif qui applique des commandes ligne par ligne au flux d'entrée
- **Syntaxe** : sed [options] 'script' [fichier] ou sed [options] -f script.sed [fichier]

Les modifications sont temporaires :

le résultat part sur la sortie standard, le fichier original est intact

- Le script est la liste des commandes à exécuter sur chaque ligne
- Sans fichier, sed lit l'entrée standard, donc s'insère dans les pipelines
- Le script se met entre guillemets simples pour le protéger du shell

La substitution s/// et le drapeau g

s (substitute): remplacer un motif spécifique

- **s/motif/remplacement/** : remplace la première occurrence du motif sur chaque ligne
- **s/motif/remplacement/g** : remplace toutes les occurrences (**g** pour global)
- **Exemple** : sed 's/2012/2013/g' cal-12.txt > cal-13.txt
- **Piège** : sans **g**, seule la première occurrence de chaque ligne est remplacée
- Le motif peut être une regex complète vue à la section 1

Substitution : & et séparateurs

- `&` dans le remplacement : réinsère le texte correspondant trouvé
- `sed 's/[0-9]*/[ & ]/'` entoure chaque nombre d'espaces et de crochets
- **Changer le séparateur** : `s|motif|remplacement|` quand le motif contient des `/`
- `sed 's|/usr/local|/opt|g'` remplace un chemin sans échapper les barres obliques
- On peut aussi utiliser `:` ou `#` comme séparateur
- Le choix du séparateur rend le script plus lisible

Supprimer : d

d (**delete**) : supprime les lignes sélectionnées

- sed '10,20**d**' fichier : supprime les lignes 10 à 20
- sed '/^$/**d**' fichier : supprime les lignes vides
- sed '/^#/**d**' fichier : supprime les lignes de commentaire
- sed '$**d**' fichier : supprime la dernière ligne
- **Combiner** : sed '/^#/d**;** /^$/d' fichier supprime commentaires et lignes vides

Adressage par numéros de ligne

Sans adresse, une commande s'applique à toutes les lignes

- **Adresse numérique** : `sed '5s/foo/bar/'` n'agit que sur la ligne **5**
- **Plage numérique** : `sed '10,20d'` supprime les lignes 10 à 20
- **Dernière ligne** : `$` désigne la fin — `sed '$d'` supprime la dernière ligne
- De la ligne n à la fin : `sed '5,$d'` supprime de la ligne 5 jusqu'à la fin
- L'adresse précède la commande dans le script

Afficher : p et -n

- **p** (**print**) : imprime la ligne courante — mais attention au double affichage
- **-n** (**silencieux**) : supprime l'affichage automatique, pour n'imprimer que ce qui est demandé
- sed -n '/ERROR/p' log.txt : n'affiche que les lignes contenant **ERROR**
- sed -n '10,20p' fichier : n'affiche que les lignes **10** à **20**
- L'option **-n** et la commande **p** vont toujours ensemble
- C'est le pendant de grep en mode extraction

Insérer, ajouter, changer : i, a, c

- `i\text` (**insert**) : insère une ligne de texte **avant** la ligne courante
- `a\text` (**append**) : ajoute une ligne de texte **après** la ligne courante
- `c\text` (**change**) : remplace la plage sélectionnée **par** le texte fourni
- **Exemple** : `sed '1i\#!/bin/bash' script.sh` insère le shebang en tête
- **Exemple** : `sed '/^\[Section\]$/a\# commentaire'` ajoute une ligne après un en-tête
- Ces commandes servent à insérer des en-têtes ou des marqueurs dans un fichier

Édition en place : -i, -e, -f

- **-i** (**in-place**) : modifie le fichier directement au lieu d'écrire sur la sortie standard
- `sed -i 's/[[:space:]]*$//' *.py` : supprime les espaces de fin de ligne de tous les fichiers Python
- **-i** travaille en réalité sur un fichier temporaire qu'il substitue à l'original
- **-e** : enchaîne plusieurs commandes — `sed -e 's/a/b/' -e 's/c/d/'`
- **-f** fichier : lit le script depuis un fichier de commandes
- Prudence : un **-i** mal ciblé modifie irrémédiablement les fichiers

Modificateurs de Casse dans sed : \u, \U, \l, \L

Transformer la casse du texte capturé (&) à la volée lors de la substitution.

Modificateur	Description	Exemple d'application ("iLiAs")
\u&	Première lettre en majuscule	ILiAs
\U&	Tout en majuscules	ILIAS
\l&	Première lettre en minuscule	iLiAs
\L&	Tout en minuscules	ilias

Exemple: sed 's/^[a-z]*/\u&/' fichier.txt

Rôle : Met la première lettre de chaque ligne en majuscule tout en conservant le reste intact.

Exemple : nettoyer un CSV - Virgules vers tabulations

Un fichier CSV sépare ses colonnes par des virgules ; il faut parfois le réparer

1. **Virgules vers tabulations** : sed 's/,/\t/g' data.csv
   - **s/** : La commande de substitution (remplacement).
   - **,** (le premier élément entre les slashes) : C'est le motif recherché : ici, une simple virgule.
   - \t (le deuxième élément) : C'est le texte de remplacement : \t représente une tabulation.
   - g (le drapeau à la fin) : Pour **Global**. Cela indique à sed de remplacer toutes les virgules trouvées sur la ligne, et pas seulement la première.
   - data.csv : Le fichier cible sur lequel la commande s'applique.

Exemple : nettoyer un CSV - Supprimer les espaces de fin de ligne

2. **Supprimer les espaces de fin de ligne :** sed 's/[[:space:]]*$//' data.csv
   - `s/` : La commande de substitution (remplacement).
   - `[[:space:]]` : Une classe de caractères POSIX qui représente n'importe quel type d'espace (un espace classique, une tabulation, etc.). C'est plus large et plus propre que de mettre juste un espace entre crochets.
   - `*` : [[:space:]]* cible une suite d'espaces s'il y en a.
   - `$` : Une ancre qui désigne la fin de la ligne.
   - `//`: Vu qu'il n'y a rien entre les deux slashes, cela signifie qu'on remplace tout ce qui a été trouvé par du vide. Les espaces de fin de ligne sont donc purement et simplement supprimés.

Exemple : nettoyer un CSV - Passer en majuscules

2. Passer en majuscules : sed 's/.*/\U&/' ***data.csv***

- s/ : La commande de substitution (remplacement).
- /.*/ : Le motif recherché :
  - . (le point) signifie « n'importe quel caractère ». * signifie « zéro, une ou plusieurs fois ».
  - .* correspond à la ligne entière (du début à la fin).
- \U& : Le texte de remplacement :
  - & représente tout ce qui a été trouvé par le motif (c'est-à-dire la ligne entière).
  - \U indique à sed de convertir le texte qui suit en majuscules (Uppercase).

Exemple : Normaliser les séparateurs d’un fichier de conf

Les fichiers de configuration gagnent à être normalisés avant d'être relus

Normaliser les séparateurs : sed 's/ *= */=/g' ***fichier.conf***

1. *= * : Le motif recherché (situé entre le 1er et 2e slash) :
   - (un espace) suivi de *: Zéro, un ou plusieurs espaces avant le signe égal.
   - = : Le signe égal littéral.
   - (un espace) suivi de *: Zéro, un ou plusieurs espaces après le signe égal.
2. /=/ : Le texte de remplacement (situé entre le 2e et 3e slash) :On remplace le tout par un signe = seul, sans aucun espace autour.
3. g (Global) : Applique le remplacement à toutes les occurrences sur chaque ligne (utile s'il y a plusieurs affectations sur la même ligne)

Exemple : Supprimer les commentaires d’un fichier de conf

Supprimer les commentaires : sed '/^#/**d**' ***fichier.conf***

- /^#/ : Le motif de sélection (situé entre les slashes) :
  - ^ : Une ancre qui représente le début de la ligne.
  - # : Le caractère dièse littéral.
  - **Ensemble** (^#) : Indique qu'on cible uniquement les lignes qui commencent par un #.
- **d** : Elle indique à sed de supprimer complètement les lignes qui correspondent au motif trouvé (donc les lignes de commentaires).
- ***fichier.conf*** : Le fichier cible sur lequel s'applique la commande.

Exemple : convertir CRLF vers LF

Windows et Unix marquent différemment la fin des lignes

- **CRLF**: Carriage Return and Line Feed – **LF**: Line Feed
- **Windows (DOS)** : `\r\n` — retour chariot puis saut de ligne
- **Unix/Linux** : `\n` — saut de ligne seul
- Le `\r` parasite apparaît dans les scripts transférés depuis Windows
- Convertir DOS → Unix : sed 's/\r$//' fichier.txt
- On cible le retour chariot en fin de ligne avec l'ancre $

Quiz 2 : sed

Q1. Que fait sed 's/foo/bar/' fichier ?

- A. remplace la première occurrence de foo sur chaque ligne
- B. remplace toute les occurrences de foo sur chaque ligne. C. modifie le fichier sur place

Q2. Comment rendre la substitution globale ?

- A. ajouter -g à la commande · B. ajouter -i ·
- C. ajouter le drapeau g au script : **s/foo/bar/g**

Q3. Que fait sed '10,20d' fichier ?

- A. affiche les lignes 10 à 20 · B. supprime les lignes 10 à 20
- C. remplace les lignes 10 à 20 · D. duplique les lignes 10 à 20

Q4. Pourquoi coupler l'option -n et la commande p ?

- A. pour n'imprimer que les lignes demandées · B. pour supprimer des lignes ·
- C. pour lire un script

Section 4 : déboguer un pipeline avec tee

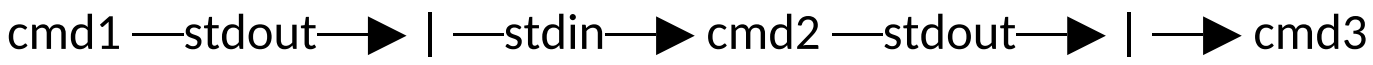
Rappel, les flux et les pipes

Le problème:

- Un pipeline enchaîne plusieurs commandes : cmd1 | cmd2 | cmd3 | ...
- On ne voit que le résultat final
- Quand il est faux ou vide, quelle étape est fautive ?

```
~$ cat access.log | grep " 404 " | cut -d' ' -f1 | sort | uniq -c | sort -rn | head -5
# Résultat : rien... pourquoi ?
```

Rappel, les flux et les pipes:



- **Les données intermédiaires** ne sont jamais affichées ni sauvegardées

Présentation de tee

tee lit stdin, écrit dans un fichier ET réémet sur stdout

- Comme un raccord en T en plomberie : le flux continue, et on en prélève une copie



- **Stratégie de débogage:**

1. Insérer un tee après chaque étape suspecte
2. Exécuter le pipeline une fois
3. Inspecter les fichiers intermédiaires : head, wc -l, less
4. Repérer la première étape où les données deviennent incorrectes
5. Corriger, puis retirer les tee

Exemple de base

Fichier access.log :

```
192.168.1.10 - - [10/Oct/2026] "GET /index.html"
200 192.168.1.22 - - [10/Oct/2026] "GET /admin"
404 192.168.1.10 - - [10/Oct/2026] "GET /old"
404 192.168.1.22 - - [10/Oct/2026] "GET /test" 404
```

Objectif : les 5 IP qui produisent le plus d'erreurs 404.

```
cat access.log \
  | tee /tmp/1_brut.txt \
  | grep " 404" \
  | tee /tmp/2_apres_grep.txt \
  | cut -d' ' -f1 \
  | tee /tmp/3_apres_cut.txt \
  | sort \
  | uniq -c \
  | sort -rn \
  | head -5
```

Quiz: Outils de Traitement de Texte



https://forms.cloud.microsoft/r/jNjhjt9zgw